

Web Programming & Python - Assignment 7 & 8

Admission Number: U24AI055

1. Linked List Implementation

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node

    def delete(self, key):
        temp = self.head
        if temp and temp.data == key:
            self.head = temp.next
            return
        while temp and temp.next and temp.next.data != key:
            temp = temp.next
        if temp and temp.next:
            temp.next = temp.next.next

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=' -> ')
            temp = temp.next
        print("None")
```

2. Queue Implementation

```
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)
```

```
def dequeue(self):
    if self.is_empty():
        return "Queue is empty"
    return self.queue.pop(0)

def is_empty(self):
    return len(self.queue) == 0
```

3. Bank Account Management

```
class Bank:
    def __init__(self):
        self.accounts = {}

    def create_account(self, acc_no, balance=0):
        self.accounts[acc_no] = balance

    def deposit(self, acc_no, amount):
        self.accounts[acc_no] += amount

    def withdraw(self, acc_no, amount):
        if self.accounts[acc_no] >= amount:
            self.accounts[acc_no] -= amount
        else:
            print("Insufficient funds")
```

4. Employee Class with Operator Overloading

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def __add__(self, other):
        return self.salary + other.salary

    def __sub__(self, other):
        return self.salary - other.salary
```

5. Shape Class with Derived Classes

```
class Shape:
    def area(self):
        pass

    def perimeter(self):
        pass
```

```
class Rectangle(Shape):
    def __init__(self, length, width):
```

```

    self.length = length
    self.width = width

def area(self):
    return self.length * self.width

def perimeter(self):
    return 2 * (self.length + self.width)

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

    def perimeter(self):
        return 2 * 3.14 * self.radius

```

6. BankAccount Class

```

class BankAccount:
    def __init__(self, acc_no, balance):
        self.acc_no = acc_no
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
        else:
            print("Insufficient funds")

    def display(self):
        print(f"Account No: {self.acc_no}, Balance: {self.balance}")

```

7. 2D and 3D Vector Operations

```

import math

class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def magnitude(self):
        return math.sqrt(self.x**2 + self.y**2)

    def rotate(self, angle):
        rad = math.radians(angle)

```

```

new_x = self.x * math.cos(rad) - self.y * math.sin(rad)
new_y = self.x * math.sin(rad) + self.y * math.cos(rad)
return Vector2D(new_x, new_y)

```

8. Message Decoder

```

def decode_message(message, output=""):
    if not message:
        print(output)
        return

    if message[0] != '0':
        decode_message(message[1:], output + chr(int(message[0]) + 64))

    if len(message) > 1 and '10' <= message[:2] <= '26':
        decode_message(message[2:], output + chr(int(message[:2]) + 64))

message = "11106"
decode_message(message)

```

9. Custom Tokenizer

```

import re

def tokenizer(text):
    pattern = r"[\w.-]+@[ \w.-.]+\b(?:https?://\w\w\.)\S+|\d+(?:[.]/\d+)*[[@#]\w+|\w+"
    return re.findall(pattern, text)

text = "Email me at example@mail.com or visit https://example.com!"
print(tokenizer(text))

```